

BAB 8

ERROR HANDLING DAN JAVA I/O

8.1 Pendahuluan

Pada bab sebelumnya, mahasiswa telah mempelajari relasi antar object, yaitu bagaimana beberapa object dapat saling berhubungan dalam sebuah program. Materi tersebut penting karena program yang baik tidak hanya terdiri dari satu class, tetapi terdiri dari beberapa class yang saling bekerja sama.

Pada BAB 8 ini, mahasiswa akan mempelajari dua materi penting, yaitu **error handling** dan **Java I/O**. Error handling digunakan untuk menangani kesalahan program agar program tidak langsung berhenti, sedangkan Java I/O digunakan untuk membaca dan menulis data, misalnya dari keyboard atau file.

Dalam praktik pemrograman, program tidak selalu berjalan sesuai harapan. Pengguna bisa salah memasukkan input, file yang dibaca bisa tidak ditemukan, atau program bisa mengalami kesalahan saat melakukan operasi tertentu. Oleh karena itu, programmer perlu membuat program yang mampu menghadapi kondisi tersebut dengan baik.

8.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu:

1. Menjelaskan pengertian error dan exception dalam Java.
2. Membedakan syntax error, runtime error, dan logical error.
3. Menggunakan blok `try-catch` untuk menangani exception.
4. Menggunakan blok `finally` dalam program.
5. Memahami penggunaan sederhana `throw` dan `throws`.
6. Membaca input dari pengguna menggunakan `Scanner`.
7. Menulis data ke file teks menggunakan Java I/O.
8. Membaca data dari file teks menggunakan Java I/O.
9. Membuat program sederhana yang menangani error dan menyimpan data ke file.

8.3 Pengertian Error dan Exception

Dalam pemrograman, **error** adalah kesalahan yang menyebabkan program tidak berjalan sesuai harapan. Kesalahan ini bisa muncul karena penulisan kode yang salah, input yang tidak sesuai, atau logika program yang keliru.

Dalam Java, kesalahan yang terjadi saat program sedang berjalan disebut **exception**. Exception dapat ditangani menggunakan mekanisme error handling agar program tidak langsung berhenti secara tiba-tiba.

Contoh kondisi yang dapat menyebabkan exception:

- pengguna memasukkan huruf saat program meminta angka
- program membagi angka dengan nol
- file yang ingin dibaca tidak ditemukan
- indeks array yang diakses melebihi batas
- data kosong tetapi tetap diproses oleh program

Contoh exception sederhana:

```
public class ContohException {
    public static void main(String[] args) {
        int hasil = 10 / 0;
        System.out.println("Hasil: " + hasil);
    }
}
```

Program di atas akan error saat dijalankan karena pembagian dengan nol tidak diperbolehkan. Kesalahan tersebut disebut `ArithmeticException`.

8.4 Jenis-Jenis Kesalahan dalam Program

Secara sederhana, kesalahan dalam program dapat dibagi menjadi tiga jenis utama. Ketiga jenis kesalahan ini perlu dipahami agar mahasiswa dapat membedakan apakah kesalahan terjadi karena penulisan kode, proses eksekusi, atau logika program.

8.4.1 Syntax Error

Syntax error adalah kesalahan yang terjadi karena penulisan kode tidak sesuai aturan bahasa Java. Kesalahan ini biasanya langsung terdeteksi oleh NetBeans sebelum program dijalankan.

Contoh syntax error:

```
public class ContohSyntaxError {
    public static void main(String[] args) {
        System.out.println("Halo ITERA")
    }
}
```

Kode di atas salah karena tidak ada tanda titik koma (;) setelah perintah `System.out.println()`. Kesalahan seperti ini harus diperbaiki terlebih dahulu sebelum program dapat dijalankan.

8.4.2 Runtime Error

Runtime error adalah kesalahan yang muncul saat program sedang dijalankan. Program mungkin berhasil dikompilasi, tetapi berhenti ketika menemukan kondisi tertentu yang tidak dapat diproses.

Contoh runtime error:

```
public class ContohRuntimeError {
    public static void main(String[] args) {
        int angka = 10;
        int pembagi = 0;

        int hasil = angka / pembagi;
        System.out.println("Hasil: " + hasil);
    }
}
```

Program tersebut tidak memiliki kesalahan penulisan kode, tetapi akan error saat dijalankan. Hal ini terjadi karena program mencoba melakukan pembagian dengan nol.

8.4.3 Logical Error

Logical error adalah kesalahan logika dalam program. Program dapat berjalan tanpa error, tetapi hasil yang diberikan tidak sesuai dengan yang diharapkan.

Contoh logical error:

```
public class ContohLogicalError {
    public static void main(String[] args) {
        int panjang = 10;
        int lebar = 5;

        int luas = panjang + lebar;

        System.out.println("Luas persegi panjang: " + luas);
    }
}
```

Program di atas dapat berjalan, tetapi hasilnya salah. Untuk menghitung luas persegi panjang seharusnya menggunakan perkalian, bukan penjumlahan.

8.5 Exception Handling dengan `try-catch`

Exception handling adalah mekanisme untuk menangani kesalahan saat program berjalan. Dengan menggunakan exception handling, program tidak langsung berhenti ketika terjadi kesalahan, tetapi dapat memberikan pesan yang lebih jelas kepada pengguna.

Dalam Java, struktur dasar exception handling menggunakan blok `try-catch`. Kode yang berpotensi menimbulkan error diletakkan di dalam blok `try`, sedangkan penanganan error diletakkan di dalam blok `catch`.

Bentuk umum `try-catch`:

```
try {
    // kode yang berpotensi error
} catch (JenisException e) {
    // kode untuk menangani error
}
```

Contoh program:

```

public class DemoTryCatch {
    public static void main(String[] args) {

        try {
            int angka = 10;
            int pembagi = 0;
            int hasil = angka / pembagi;

            System.out.println("Hasil: " + hasil);
        } catch (ArithmeticException e) {
            System.out.println("Terjadi kesalahan: tidak bisa membagi
dengan nol.");
        }

        System.out.println("Program selesai dijalankan.");
    }
}

```

Output:

```

Terjadi kesalahan: tidak bisa membagi dengan nol.
Program selesai dijalankan.

```

Pada contoh tersebut, program tidak langsung berhenti meskipun terjadi kesalahan pembagian dengan nol. Kesalahan ditangani oleh blok `catch`, kemudian program tetap melanjutkan perintah berikutnya.

8.6 Menangani Input yang Salah

Dalam praktikum, mahasiswa sering membuat program yang meminta input dari pengguna. Masalah dapat terjadi ketika program meminta angka, tetapi pengguna memasukkan huruf atau teks.

Untuk menangani kondisi tersebut, Java menyediakan exception `InputMismatchException`. Exception ini biasanya muncul ketika tipe input tidak sesuai dengan tipe data yang diminta.

Contoh program tanpa error handling:

```

import java.util.Scanner;

public class InputTanpaHandling {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();

        System.out.println("Nilai Anda: " + nilai);
    }
}

```

Jika pengguna memasukkan teks seperti delapan puluh, program akan error. Agar lebih aman, program dapat diperbaiki seperti berikut.

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class InputDenganHandling {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        try {
            System.out.print("Masukkan nilai: ");
            int nilai = input.nextInt();

            System.out.println("Nilai Anda: " + nilai);
        } catch (InputMismatchException e) {
            System.out.println("Input salah. Nilai harus berupa
angka.");
        }

        System.out.println("Program selesai.");
    }
}
```

Output jika input salah:

```
Masukkan nilai: delapan puluh
Input salah. Nilai harus berupa angka.
Program selesai.
```

Dengan cara ini, program menjadi lebih ramah terhadap kesalahan pengguna. Program tidak langsung berhenti, tetapi memberikan pesan yang mudah dipahami.

8.7 Blok `finally`

Blok `finally` adalah bagian dari exception handling yang akan selalu dijalankan, baik terjadi exception maupun tidak. Blok ini biasanya digunakan untuk menutup file, menutup koneksi, atau menampilkan pesan akhir.

Bentuk umum:

```
try {
    // kode yang berpotensi error
} catch (Exception e) {
    // penanganan error
} finally {
    // kode yang selalu dijalankan
}
```

Contoh program:

```

public class DemoFinally {
    public static void main(String[] args) {

        try {
            int hasil = 10 / 2;
            System.out.println("Hasil: " + hasil);
        } catch (ArithmeticException e) {
            System.out.println("Terjadi kesalahan pembagian.");
        } finally {
            System.out.println("Blok finally tetap dijalankan.");
        }

        System.out.println("Program selesai.");
    }
}

```

Output:

```

Hasil: 5
Blok finally tetap dijalankan.
Program selesai.

```

Jika pembagiannya diubah menjadi nol, blok `finally` tetap dijalankan. Hal ini menunjukkan bahwa `finally` berguna untuk memastikan bagian tertentu dari program tetap berjalan.

8.8 Keyword `throw` dan `throws`

Selain menangani exception yang sudah terjadi, Java juga memungkinkan programmer untuk membuat exception secara manual. Dua keyword yang sering digunakan adalah `throw` dan `throws`.

Keyword `throw` digunakan untuk melempar exception secara manual. Biasanya ini digunakan ketika programmer ingin membuat aturan validasi tertentu.

Contoh penggunaan `throw`:

```

public class DemoThrow {
    static void cekNilai(int nilai) {
        if (nilai < 0) {
            throw new IllegalArgumentException("Nilai tidak boleh negatif.");
        } else {
            System.out.println("Nilai valid: " + nilai);
        }
    }

    public static void main(String[] args) {
        cekNilai(80);
        cekNilai(-10);
    }
}

```

Pada program tersebut, jika nilai kurang dari nol, program akan melempar exception. Ini berguna untuk menjaga agar data yang masuk tetap sesuai aturan.

Keyword `throws` digunakan pada deklarasi method untuk memberi tahu bahwa method tersebut mungkin menghasilkan exception. Biasanya `throws` sering digunakan ketika bekerja dengan file.

Contoh sederhana:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class DemoThrows {

    static void bacaFile() throws FileNotFoundException {
        File file = new File("data.txt");
        Scanner baca = new Scanner(file);

        while (baca.hasNextLine()) {
            System.out.println(baca.nextLine());
        }

        baca.close();
    }

    public static void main(String[] args) {
        try {
            bacaFile();
        } catch (FileNotFoundException e) {
            System.out.println("File tidak ditemukan.");
        }
    }
}
```

Pada contoh tersebut, method `bacaFile()` diberi `throws FileNotFoundException`. Artinya, method tersebut berpotensi menghasilkan error jika file tidak ditemukan.

8.9 Pengenalan Java I/O

Java I/O adalah fitur Java yang digunakan untuk proses **input** dan **output**. Input berarti data masuk ke program, sedangkan output berarti data keluar dari program.

Contoh input:

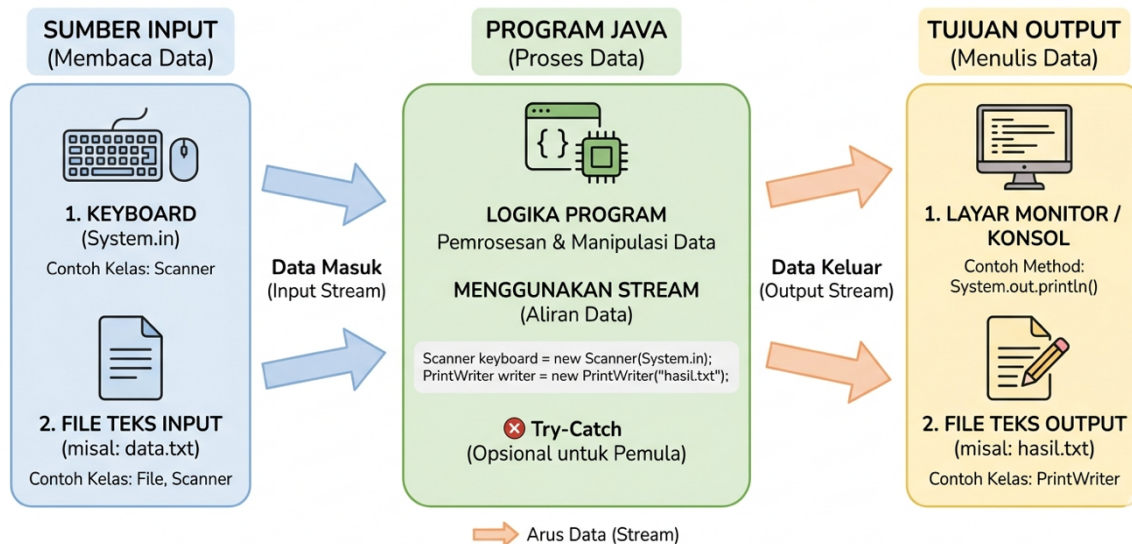
- data dari keyboard
- data dari file
- data dari database

Contoh output:

- teks yang tampil di layar
- data yang disimpan ke file
- data yang dikirim ke printer atau sistem lain

Pada bab ini, pembahasan difokuskan pada input dari keyboard dan file teks. Materi ini penting karena pada mini project, program biasanya perlu menyimpan data agar tidak hilang setelah program ditutup.

DIAGRAM KONSEP JAVA I/O (INPUT/OUTPUT)



Gambar 8.1 Ilustrasi Konsep Java I/O

Gambar ini menunjukkan alur input dan output dalam Java. Data dapat masuk ke program melalui keyboard atau file, kemudian hasilnya dapat ditampilkan ke layar atau disimpan kembali ke file.

8.10 Membaca Input dengan `Scanner`

Class `Scanner` digunakan untuk membaca input dari pengguna melalui keyboard. Class ini sudah sering digunakan pada program dasar Java, tetapi pada bab ini penggunaannya dikaitkan dengan error handling.

Contoh membaca input:


```
import java.util.Scanner;

public class DemoScanner {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nama: ");
        String nama = input.nextLine();

        System.out.print("Masukkan umur: ");
        int umur = input.nextInt();

        System.out.println("Nama: " + nama);
        System.out.println("Umur: " + umur);
    }
}
```

Beberapa method yang sering digunakan pada Scanner:

Method	Fungsi
<code>nextLine()</code>	membaca teks satu baris
<code>next()</code>	membaca satu kata
<code>nextInt()</code>	membaca bilangan bulat
<code>nextDouble()</code>	membaca bilangan desimal

Kesalahan umum saat memakai Scanner adalah mencampur `nextInt()` dan `nextLine()` tanpa membersihkan baris input. Jika setelah `nextInt()` ingin membaca teks dengan `nextLine()`, biasanya perlu menambahkan `input.nextLine()` untuk membuang enter yang tersisa.

Contoh:

```
import java.util.Scanner;

public class ScannerNextLine {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan umur: ");
        int umur = input.nextInt();
        input.nextLine(); // membersihkan enter

        System.out.print("Masukkan nama lengkap: ");
        String nama = input.nextLine();

        System.out.println("Nama: " + nama);
        System.out.println("Umur: " + umur);
    }
}
```

8.11 Menulis Data ke File

Java dapat digunakan untuk menyimpan data ke dalam file teks. Salah satu cara sederhana adalah menggunakan `FileWriter` dan `PrintWriter`.

Contoh program menulis data ke file:

```
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.IOException;

public class TulisFile {
    public static void main(String[] args) {

        try {
            FileWriter fileWriter = new FileWriter("mahasiswa.txt");
            PrintWriter printWriter = new PrintWriter(fileWriter);

            printWriter.println("Nama : Andi Saputra");
            printWriter.println("NIM : 123140001");
            printWriter.println("Prodi: Informatika");

            printWriter.close();

            System.out.println("Data berhasil ditulis ke file.");
        } catch (IOException e) {
            System.out.println("Terjadi kesalahan saat menulis file.");
        }
    }
}
```

Jika program berhasil dijalankan, maka file `mahasiswa.txt` akan dibuat di folder project. Isi file tersebut adalah data mahasiswa yang ditulis melalui program.

Jika ingin menambahkan data baru tanpa menghapus isi file lama, gunakan mode append dengan menambahkan `true` pada `FileWriter`.

```
FileWriter fileWriter = new FileWriter("mahasiswa.txt", true);
```

Mode append berguna ketika program ingin menyimpan banyak data secara bertahap. Misalnya, setiap kali pengguna menginput data mahasiswa baru, data tersebut ditambahkan ke bagian akhir file.

8.12 Membaca Data dari File

Selain menulis file, Java juga dapat membaca isi file. Salah satu cara sederhana adalah menggunakan class `File` dan `Scanner`.

Contoh program membaca file:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class BacaFile {
    public static void main(String[] args) {

        try {
            File file = new File("mahasiswa.txt");
            Scanner baca = new Scanner(file);

            while (baca.hasNextLine()) {
                String data = baca.nextLine();
                System.out.println(data);
            }

            baca.close();
        } catch (FileNotFoundException e) {
            System.out.println("File tidak ditemukan.");
        }
    }
}
```

Program tersebut akan membaca file `mahasiswa.txt` baris demi baris. Jika file tidak ditemukan, program tidak langsung berhenti, tetapi menampilkan pesan bahwa file tidak ditemukan.

8.13 Studi Kasus Praktikum: Data Mahasiswa dengan File

Pada studi kasus ini, mahasiswa akan membuat program sederhana untuk menyimpan data mahasiswa ke file dan membaca kembali isi file tersebut. Program juga menggunakan error handling agar lebih aman saat menerima input dan mengakses file.

```
import java.io.File;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.IOException;
import java.io.FileNotFoundException;
import java.util.Scanner;
import java.util.InputMismatchException;

public class SistemDataMahasiswa {

    static void simpanData(String nama, String nim, String prodi) {
        try {
            FileWriter fileWriter = new FileWriter("data_mahasiswa.txt",
true);
            PrintWriter printWriter = new PrintWriter(fileWriter);

            printWriter.println("Nama : " + nama);
            printWriter.println("NIM : " + nim);
            printWriter.println("Prodi: " + prodi);
            printWriter.println("-----");

            printWriter.close();

            System.out.println("Data mahasiswa berhasil disimpan.");
        } catch (IOException e) {
            System.out.println("Terjadi kesalahan saat menyimpan data.");
        }
    }

    static void bacaData() {
        try {
            File file = new File("data_mahasiswa.txt");
            Scanner baca = new Scanner(file);

            System.out.println("\nIsi File Data Mahasiswa:");
            while (baca.hasNextLine()) {
                System.out.println(baca.nextLine());
            }

            baca.close();
        } catch (FileNotFoundException e) {
            System.out.println("File data_mahasiswa.txt belum ditemukan.");
        }
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        try {
            System.out.println("=== Program Data Mahasiswa ITERA ===");

            System.out.print("Masukkan nama : ");
            String nama = input.nextLine();
```

```

        System.out.print("Masukkan NIM    : ");
        String nim = input.nextLine();

        System.out.print("Masukkan prodi : ");
        String prodi = input.nextLine();

        if (nama.isEmpty() || nim.isEmpty() || prodi.isEmpty()) {
            throw new IllegalArgumentException("Data tidak boleh
kosong.");
        }

        simpanData(nama, nim, prodi);
        bacaData();

    } catch (IllegalArgumentException e) {
        System.out.println("Kesalahan input: " + e.getMessage());
    } finally {
        System.out.println("\nProgram selesai dijalankan.");
    }
}
}

```

8.13.1 Contoh Output

=== Program Data Mahasiswa ITERA ===

Masukkan nama : Andi Saputra

Masukkan NIM : 123140001

Masukkan prodi : Informatika

Data mahasiswa berhasil disimpan.

Isi File Data Mahasiswa:

Nama : Andi Saputra

NIM : 123140001

Prodi: Informatika

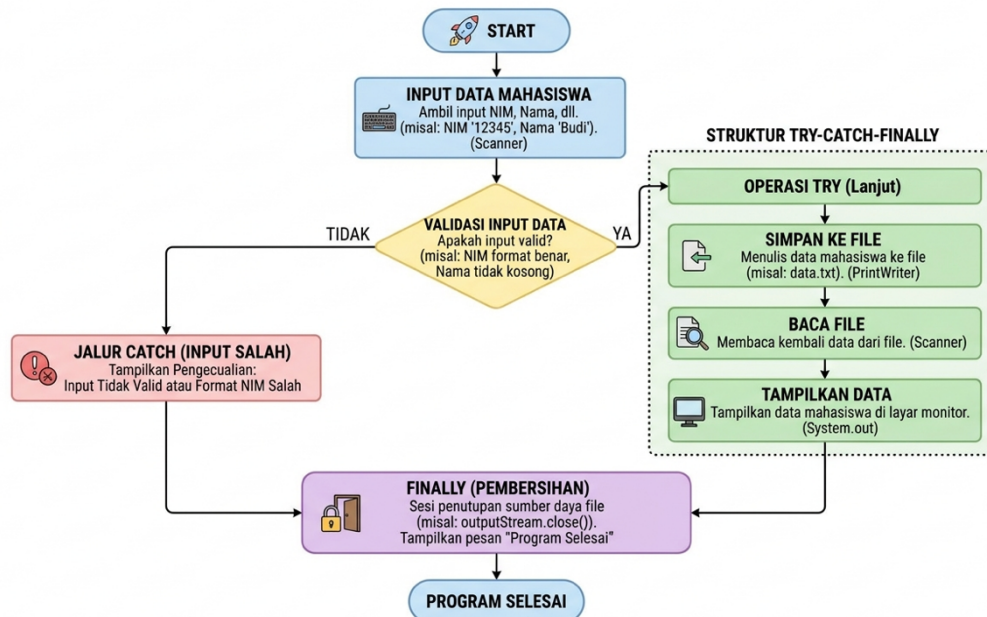
Program selesai dijalankan.

Pada program tersebut, method `simpanData()` digunakan untuk menyimpan data ke file. Method `bacaData()` digunakan untuk membaca kembali isi file.

Program juga menggunakan `try-catch` untuk menangani kemungkinan kesalahan saat menulis dan membaca file. Selain itu, program menggunakan `throw` untuk menolak data kosong.

8.14 Alur Error Handling dan Java I/O

Agar mahasiswa lebih mudah memahami alur kerja error handling dan file, visualisasi dapat ditambahkan pada bagian ini. Gambar sebaiknya menjelaskan bahwa program menerima input, memvalidasi data, menyimpan data ke file, lalu membaca data kembali.



MODUL PRAKTIKUM JAVA

Gambar 8.2 Alur Error Handling dan Java I/O pada Program Data Mahasiswa

Gambar ini menunjukkan alur program saat menerima input, memvalidasi data, menyimpan data ke file, membaca kembali isi file, serta menangani kesalahan menggunakan `try-catch-finally`.

8.15 Kesalahan Umum dalam Error Handling dan Java I/O

Saat belajar error handling dan Java I/O, mahasiswa sering melakukan beberapa kesalahan. Kesalahan ini wajar karena mahasiswa mulai berinteraksi dengan input pengguna dan file eksternal.

Beberapa kesalahan umum adalah:

1. **Tidak menggunakan `try-catch` saat membaca file**

Membaca file dapat menyebabkan error jika file tidak ditemukan, sehingga perlu ditangani.

2. **Salah menulis nama file**

Nama file harus sama dengan file yang akan dibaca atau ditulis, termasuk ekstensi seperti `.txt`.

3. **Lupa menutup file**

File sebaiknya ditutup setelah selesai digunakan agar data tersimpan dengan benar dan resource tidak terpakai terus.

4. Program berhenti karena input tidak sesuai tipe data

Jika program meminta angka, tetapi pengguna memasukkan teks, program dapat menghasilkan exception.

5. Menangkap exception terlalu umum tanpa memahami penyebabnya

Menangkap `Exception` memang mudah, tetapi mahasiswa tetap perlu memahami jenis kesalahan yang terjadi.

6. Tidak tahu lokasi file tersimpan

Pada NetBeans, file biasanya tersimpan di folder project. Mahasiswa perlu mengecek folder project untuk menemukan file hasil program.

8.16 Tugas Percobaan

Percobaan 1: Try-Catch Dasar

Buat program pembagian dua angka dengan ketentuan berikut:

1. Program meminta input dua angka dari pengguna.
2. Program membagi angka pertama dengan angka kedua.
3. Gunakan `try-catch` untuk menangani pembagian dengan nol.
4. Tampilkan pesan error jika pembagi bernilai nol.
5. Tampilkan hasil pembagian jika tidak terjadi error.

Percobaan 2: Validasi Input Nilai

Buat program input nilai mahasiswa dengan ketentuan berikut:

1. Program meminta input nilai mahasiswa.
2. Gunakan `try-catch` untuk menangani input yang bukan angka.
3. Jika nilai valid, tampilkan nilai tersebut.
4. Jika input salah, tampilkan pesan bahwa input harus berupa angka.
5. Tambahkan validasi bahwa nilai tidak boleh kurang dari 0 dan tidak boleh lebih dari 100.

Percobaan 3: Menulis dan Membaca File

Buat program untuk menyimpan data mahasiswa ke file teks dengan ketentuan berikut:

1. Program meminta input nama, NIM, dan prodi.
2. Simpan data tersebut ke file `mahasiswa.txt`.
3. Setelah data disimpan, baca kembali isi file tersebut.
4. Gunakan `try-catch` untuk menangani kesalahan saat menulis atau membaca file.
5. Tampilkan pesan jika file tidak ditemukan atau gagal ditulis.

Percobaan 4: Studi Kasus Mini

Buat program **Sistem Data Mahasiswa** dengan ketentuan berikut:

1. Program dapat menerima input data mahasiswa.
2. Data yang dimasukkan meliputi nama, NIM, prodi, dan semester.
3. Program harus menolak data kosong.
4. Program harus menolak semester yang kurang dari 1.
5. Data mahasiswa disimpan ke file `data_mahasiswa.txt`.
6. Program dapat membaca dan menampilkan kembali isi file.
7. Gunakan `try-catch`, `finally`, dan minimal satu penggunaan `throw`.

Pertanyaan Diskusi

Jawablah pertanyaan berikut sebagai bagian dari pemahaman konsep.

1. Apa yang dimaksud dengan exception dalam Java?
2. Apa perbedaan syntax error, runtime error, dan logical error?
3. Apa fungsi blok `try` dalam error handling?
4. Apa fungsi blok `catch` dalam error handling?
5. Kapan blok `finally` dijalankan?
6. Apa perbedaan `throw` dan `throws`?
7. Mengapa membaca file perlu menggunakan error handling?
8. Apa manfaat menyimpan data program ke file?

8.17 Tugas Laporan Praktikum

Setelah menyelesaikan seluruh percobaan pada BAB 8, mahasiswa diminta membuat laporan praktikum. Format laporan dibuat konsisten dengan BAB sebelumnya agar mahasiswa terbiasa dengan pola kerja yang sistematis dan rapi.

Struktur laporan:

1. Cover
2. Tujuan praktikum
3. Alat dan bahan
4. Langkah percobaan
5. Source code
6. Hasil output program
7. Analisis program
8. Kesimpulan

Ketentuan isi analisis:

Pada bagian analisis, mahasiswa diminta menjelaskan penerapan error handling dan Java I/O pada program yang dibuat. Penjelasan minimal mencakup:

- jenis exception yang ditangani
- bagian program yang menggunakan `try-catch`
- fungsi blok `finally` pada program
- penggunaan `throw` atau `throws` jika ada
- cara program menulis data ke file
- cara program membaca data dari file
- kendala yang muncul saat menjalankan program

Mahasiswa juga perlu menjelaskan bagaimana program merespons input yang salah. Penjelasan tidak perlu terlalu panjang, tetapi harus menunjukkan bahwa mahasiswa memahami cara membuat program yang lebih aman.

Laporan dikumpulkan dalam bentuk **PDF** sesuai ketentuan dosen atau asisten praktikum.

8.18 Penutup

Pada BAB 8 ini, mahasiswa telah mempelajari error handling dan Java I/O. Error handling membantu program menangani kesalahan agar tidak langsung berhenti, sedangkan Java I/O membantu program membaca dan menyimpan data.